



MERN STACK TRAINING

JavaScript Deep Dive — Session JS-2

Scope · Lexical Scope · Hoisting · TDZ · Functions · Closures · Currying

What this session covers, from first principles:

- Why scope exists and how the JS engine decides what a variable name means
- Lexical (static) scope and the scope chain
- Hoisting — the two-phase execution model (memory creation + execution)
- Temporal Dead Zone — why `let` / `const` behave differently from `var`
- Functions in depth — declarations, expressions, arrow functions, IIFEs, first-class citizens
- Closures — the "function + its lexical backpack"
- Currying — transforming multi-argument functions, one argument at a time
- 20 output-based questions with full reasoning (interview + exam style)

Table of Contents

1.	1. Roadmap for Today's Session	Basics → Advanced
2.	2. Scope — What, Why, How	Global, Function, Block
3.	3. Lexical Scope & the Scope Chain	Static vs Dynamic
4.	4. Hoisting	var, function, let/const
5.	5. Temporal Dead Zone (TDZ)	Why let/const feel "stricter"
6.	6. Functions In Depth	Declaration → Expression → Arrow → IIFE
7.	7. Closures	The lexical backpack
8.	8. Currying	One argument at a time
9.	9. Output-Based Question Bank	20 questions, fully explained
10.	10. One-Page Recap & Cheat Sheet	For quick revision
11.	11. Practice Exercises	For you to attempt

1. Roadmap for Today's Session

Before diving in, it helps students to see the shape of the whole session. Each concept below is a prerequisite for the one after it — this is intentionally a **staircase**, not a list of unrelated topics.



Why this order? You cannot truly understand *closures* without *lexical scope*. You cannot understand the weird behaviour of `var` without *hoisting*. You cannot understand why `let` throws a "cannot access before initialization" error without the *TDZ*. And you cannot understand *currying* without first being fluent in how *functions* are values in JS. Each topic is a building block for the next — this is the "why" behind the roadmap itself.

How to Get the Most Out of This

Open a browser console (or Node REPL) side-by-side with these notes. For every subtopic below, type the snippet yourself, **predict the output first**, then run it. This "predict → run → explain" loop is what makes scope/closures/hoisting stick — you'll remember the moment your prediction was wrong.

2. Scope — What, Why, How

2.1 What is Scope?

Scope is the region of your code where a particular variable or function name is *accessible* / *visible*. It answers one question that the JS engine must answer millions of times while running your program: "*When I see the identifier `x` here, which `x` do I mean?*"

Why does scope exist at all? Imagine a large codebase with 50 files, each written by a different developer. If every variable were visible everywhere (one giant global namespace), two developers naming a variable data or count would silently overwrite each other's values. Scope is the language's mechanism for **encapsulation** — it lets you reuse simple names safely, prevents accidental collisions, and lets the engine free up memory once a scope is no longer reachable (garbage collection).

2.2 The Three Kinds of Scope in JavaScript

Scope Type	Created By	Visible Inside	Applies To
Global Scope	Code written outside any function/block	Entire program (every file sharing that global object)	<code>var</code> , <code>let</code> , <code>const</code> , functions
Function Scope	Every <code>function() {}</code>	Only inside that function (and its nested functions)	<code>var</code> is function-scoped
Block Scope	Any <code>{ ... }</code> — if, for, while, or a bare block	Only inside that block	<code>let</code> and <code>const</code> are block-scoped

Example 1 — Global Scope

```
let companyName = "MyAnatomy";

function printCompany() {
  console.log(companyName); // accessible: found in the enclosing (global) scope
}
printCompany();
```

```
► CONSOLE OUTPUT
MyAnatomy
```

Example 2 — Function Scope (why var "leaks" out of blocks)

```
function checkAge() {
  if (true) {
    var isAdult = true; // var ignores the {} block – it belongs to the function
  }
  console.log(isAdult); // still accessible here!
}
checkAge();
```

```
► CONSOLE OUTPUT
true
```

Example 3 — Block Scope (let/const respect {})

```
function checkAgeModern() {
  if (true) {
    let isAdult = true;
  }
  console.log(isAdult); // ReferenceError: isAdult is not defined
}
checkAgeModern();
```

```
► CONSOLE OUTPUT
Uncaught ReferenceError: isAdult is not defined
```

Real-world scenario: Think of a college campus. The **global scope** is the whole campus — anyone can walk there. A **function scope** is like a specific classroom — only students enrolled in that class (code inside that function) can use what's written on that room's whiteboard. A **block scope** is even narrower — like a locker that only opens during one specific period (an if block); once the period (block) ends, that locker's contents are inaccessible.

2.3 Mini Practical — Try It Yourself

Live in your browser console or a file run with node `scope.js`:

```
// scope.js
let campus = "global";

function classroom() {
```

```
let campus = "classroom"; // shadows the global one
if (true) {
  let campus = "locker"; // shadows the classroom one
  console.log("Inside block:", campus);
}
console.log("Inside function:", campus);
}

classroom();
console.log("Outside everything:", campus);
```

► CONSOLE OUTPUT

```
Inside block: locker Inside function: classroom Outside everything: global
```

This is called **variable shadowing** — an inner scope can declare a variable with the same name as an outer scope, and the inner one "wins" for as long as you're inside that inner scope.

3. Lexical Scope & the Scope Chain

3.1 What does "Lexical" mean?

Lexical comes from "lexicon" — it means *relating to the words/text as written*. Lexical scope (also called **static scope**) means: **a variable's scope is decided by WHERE it is physically written in the source code, not by how or from where the function is called**. JavaScript decides scope at *author-time* (when you write the code), not at *run-time* (when you call the function).

Why does this matter? It means you — the developer reading the code — can always figure out what a variable refers to just by looking at the nesting of the code, without having to trace every possible call path at runtime. This predictability is a major reason JS (and most modern languages) chose lexical scope over dynamic scope.

3.2 The Scope Chain

When the engine can't find a variable in the current scope, it looks **outward** to the scope in which the current function was *physically defined*, then the next one out, and so on, until it reaches the global scope. This ordered lookup path is called the **scope chain**.

```
function outer() {
  let fruit = "mango";

  function inner() {
    console.log(fruit); // not found in inner() -> look in outer() -> found!
  }

  inner();
}
```

► CONSOLE OUTPUT

mango

3.3 Lexical Scope Is Set Where the Function Is DEFINED, Not Where It Is CALLED

```
let value = "global value";

function showValue() {
  console.log(value); // showValue is DEFINED in the global scope
}

function wrapper() {
  let value = "wrapper's local value";
  showValue(); // called from inside wrapper, but that does NOT change its lexical scope
}

wrapper();
```

► CONSOLE OUTPUT

global value

Common misconception: Students often expect "wrapper's local value" because `showValue()` is *called* from inside `wrapper()`. But JS uses **lexical (static)** scope, not **dynamic** scope — the scope is fixed by where `showValue` was written (global), not by who called it.

3.4 Real-World Scenario

Think of your **home address** vs your **current location**. Lexical scope is like your home address — it's fixed the moment you're "born" (defined) and never changes no matter where you travel (are called from). A postal worker (the JS engine) always delivers mail (resolves a variable) based on your permanent home address, not on which city you're currently visiting.

3.5 Mini Practical — Nested Scope Chain

```
function department() {
  let dept = "Engineering";

  function team() {
    let teamName = "Backend";

    function developer() {
      let name = "Riya";
      // developer() can see all three levels via the scope chain
      console.log(`${name} works on ${teamName} team in ${dept}`);
    }
  }
}
```

```
    developer();  
  }  
  team();  
}  
department();
```

► CONSOLE OUTPUT

Riya works on Backend team in Engineering

Note the direction: an **inner** scope can always read **outer** variables, but an **outer** scope can never reach into an inner one. Scope chains only look outward, never inward.

4. Hoisting

4.1 What is Hoisting?

Hoisting is the observable behaviour where variable and function *declarations* appear to be "moved to the top" of their scope before the code executes. In reality, nothing physically moves — this behaviour comes from how the JS engine actually runs your code in **two phases**.

4.2 The Real Mechanism: Two-Phase Execution

Every time the JS engine enters a new execution context (global code, or a function call), it does two passes:

1. **Memory Creation Phase (a.k.a. Creation Phase):** The engine scans the code and allocates memory for every variable and function *before* running a single line. `var` variables are set to `undefined`. Function *declarations* are stored in memory with their entire function body. `let` / `const` variables are also registered, but left **uninitialized** (this is the root cause of TDZ — see Section 5).
2. **Execution Phase:** The engine now runs the code line by line, assigning real values as it reaches each line.

Why does JS work this way? This two-phase model is what allows you to call a function *before* its declaration appears in the file — a very common and intentional pattern (e.g. defining `main()` at the top and its helper functions below it, for readability).

4.3 Hoisting with var

```
console.log(city); // undefined (NOT an error!)
var city = "Agra";
console.log(city); // Agra
```

```
► CONSOLE OUTPUT
undefined
Agra
```

What the engine actually "sees" after the creation phase (mental model):

```
var city;           // hoisted, auto-initialized to undefined
console.log(city);  // undefined
city = "Agra";      // assignment happens only at this line, in execution phase
console.log(city);  // Agra
```

4.4 Hoisting with Function Declarations

```
greet(); // works! function declarations are fully hoisted (with their body)

function greet() {
  console.log("Hello from a hoisted function!");
}
```

```
► CONSOLE OUTPUT
Hello from a hoisted function!
```

4.5 Function Expressions Are NOT Hoisted the Same Way

```
sayHi(); // TypeError: sayHi is not a function

var sayHi = function () {
  console.log("Hi!");
};
```

```
► CONSOLE OUTPUT
Uncaught TypeError: sayHi is not a function
```

Why the difference? Only the `var sayHi` part is hoisted (as `undefined`); the function *value* is only assigned to it at execution time, when the line `var sayHi = function(){} actually runs`. So at the point of calling `sayHi()`, `sayHi` is still `undefined` — and calling `undefined()` throws a `TypeError`.

4.6 Mini Practical

```
console.log(typeof add); // "function" - fully hoisted
console.log(typeof multiply); // "undefined" - only the var name is hoisted

function add(a, b) { return a + b; }
var multiply = function (a, b) { return a * b; };
```

► CONSOLE OUTPUT
function undefined

Declared with	Hoisted?	Initial value before its line runs
<code>var</code>	Yes	<code>undefined</code>
Function declaration	Yes, fully (with body)	Callable immediately
<code>let</code> / <code>const</code>	Yes (the name is registered), but not initialized	In the Temporal Dead Zone — accessing it throws
Function expression / arrow fn	Only the variable holding it (per var/let/const rule above)	Same as the variable type rule

5. Temporal Dead Zone (TDZ)

5.1 What is the TDZ?

The **Temporal Dead Zone** is the span of code between the **start of a scope** and the **line where a `let` / `const` variable is actually declared**. During this span, the variable exists (it was registered in the creation phase) but is **uninitialized** — accessing it throws a `ReferenceError`.

Why was TDZ introduced? With `var`, accessing a hoisted-but-not-yet-assigned variable silently gives `undefined` — a very common source of subtle bugs, because the program keeps running with a "wrong-looking" value instead of failing loudly. The creators of `let/const` (ES2015) decided that using a variable before its declaration line is almost always a programmer mistake, so the TDZ makes this fail **loudly and immediately** instead of silently.

5.2 Example

```
console.log(score); // ReferenceError: Cannot access 'score' before initialization
let score = 90;
```

```
► CONSOLE OUTPUT
Uncaught ReferenceError: Cannot access 'score' before initialization
```

Compare directly with `var`:

	<code>var</code>	<code>let</code> / <code>const</code>
Before declaration line	<code>undefined</code> (silent)	Throws <code>ReferenceError</code> (loud)
Why	Auto-initialized in creation phase	Registered but left uninitialized — sits in the TDZ until its line runs

5.3 TDZ Is Per-Scope, Not Just "Top of File"

```
let x = "outer x";

function demo() {
  console.log(x); // ReferenceError! Not "outer x"
  let x = "inner x";
}
demo();
```

```
► CONSOLE OUTPUT
Uncaught ReferenceError: Cannot access 'x' before initialization
```

Why not "outer x"? Because `demo()` has its own `let x` declared somewhere in its body. The moment the engine sees *any* `let x` inside that function's creation phase, the name `x` is "claimed" for that entire function scope from its very first line — so the outer `x` is shadowed and unreachable, and the local `x` is still in its TDZ at the point of the `console.log`.

5.4 Real-World Scenario

Think of a **reserved hotel room** that is being cleaned. The room number exists on the hotel's system the moment a booking is made (the name is "hoisted"/registered), but the door is locked — nobody can check in (read the value) — until housekeeping finishes (the declaration line executes and assigns the value). Trying to enter early doesn't give you an empty room (`undefined`); the hotel refuses you outright (`ReferenceError`).

5.5 Mini Practical

```
function checkout(cart) {
  console.log(`Applying discount of ${discount * 100}%`);
  const discount = 0.1;
}
try {
  checkout([]);
} catch (err) {
  console.log("Caught:", err.message);
}
```

```
► CONSOLE OUTPUT
Caught: Cannot access 'discount' before initialization
```

Takeaway: Always declare `let` / `const` variables at the top of the block/scope where you plan to use them. This isn't just style — it eliminates an entire category of TDZ bugs.

6. Functions In Depth

6.1 What is a Function? (First Principles)

A function is a **reusable block of code** that takes input (parameters), performs an action, and optionally returns output. In JavaScript, functions are also **values** — they can be stored in variables, passed as arguments, and returned from other functions. This property is called being a **first-class citizen**, and it is the foundation for closures, callbacks, and currying.

Why does "functions are values" matter so much? It means you can treat behaviour itself as data — pass a piece of logic around your program just like you'd pass a number or a string. This single idea powers array methods like `.map()`/`.filter()`, event handlers, Promises, React components, and Express middleware — practically all of modern JS.

6.2 Function Declaration

```
function add(a, b) {  
  return a + b;  
}  
console.log(add(2, 3));
```

► CONSOLE OUTPUT

5

Trait: Fully hoisted (Section 4.4) — usable before its definition line.

6.3 Function Expression

```
const subtract = function (a, b) {  
  return a - b;  
};  
console.log(subtract(5, 2));
```

► CONSOLE OUTPUT

3

Trait: Only hoisted as far as its variable (Section 4.5) — not usable before its definition line.

6.4 Arrow Functions

```
const multiply = (a, b) => a * b;  
console.log(multiply(4, 5));
```

► CONSOLE OUTPUT

20

Key difference — this binding: Arrow functions do not have their own `this`; they lexically inherit `this` from the scope they were *written* in (again — lexical scoping, but this time for `this`). Regular functions get their own `this` depending on *how they're called*. This is why arrow functions are preferred for callbacks inside class methods or object methods.

```
const trainer = {  
  name: "Amit",  
  regularGreet: function () {  
    setTimeout(function () {  
      console.log("Regular fn this.name:", this.name); // 'this' lost -> undefined  
    }, 10);  
  },  
  arrowGreet: function () {  
    setTimeout(() => {  
      console.log("Arrow fn this.name:", this.name); // 'this' inherited from arrowGreet -> "Amit"  
    }, 10);  
  }  
};  
trainer.regularGreet();  
trainer.arrowGreet();
```

► CONSOLE OUTPUT

Regular fn this.name: undefined Arrow fn this.name: Amit

6.5 IIFE — Immediately Invoked Function Expression

```
(function () {
  console.log("I run immediately, once, and stay private!");
})();
```

► CONSOLE OUTPUT

I run immediately, once, and stay private!

Why use an IIFE? It creates a private function scope on the spot, so any variables declared inside never leak into the global scope. This was the standard way to avoid polluting globals before ES2015 modules and `let/const` block scoping existed — and it's still used for one-off setup code.

6.6 Default and Rest Parameters

```
function createUser(name, role = "Student") {
  return `${name} - ${role}`;
}
console.log(createUser("Priya"));           // role defaults
console.log(createUser("Rahul", "Trainer")); // role overridden

function sumAll(...numbers) { // rest parameter collects remaining args into an array
  return numbers.reduce((total, n) => total + n, 0);
}
console.log(sumAll(1, 2, 3, 4));
```

► CONSOLE OUTPUT

Priya - Student Rahul - Trainer 10

6.7 Higher-Order Functions

A **higher-order function** either (a) accepts another function as an argument, or (b) returns a function. This is only possible because functions are first-class values.

```
function withLogging(fn) {           // (a) accepts a function
  return function (...args) {        // (b) returns a function
    console.log("Calling with:", args);
    return fn(...args);
  };
}

const loggedAdd = withLogging(add);
console.log(loggedAdd(10, 20));
```

► CONSOLE OUTPUT

Calling with: [10, 20] 30

This pattern — a function returning a function — is the direct bridge into our next two topics: [Closures](#) and [Currying](#).

6.8 Real-World Scenario

In an MERN app, an Express **middleware** is a higher-order function pattern: `app.use((req, res, next) => { ... })`. React's `useCallback(() => {...}, [deps])` stores a function as a value. Array utilities like `orders.filter(o => o.status === "paid")` pass a function as an argument. Once students see functions as ordinary values, these patterns stop looking like "magic" and start looking like simple function composition.

6.9 Mini Practical

```
// A tiny "calculator factory" using first-class functions
function operate(operator) {
  const ops = {
    "+": (a, b) => a + b,
    "-": (a, b) => a - b,
    "*": (a, b) => a * b,
  };
  return ops[operator]; // returning a function chosen at runtime
}

const doAdd = operate("+");
console.log(doAdd(6, 4)); // 10
console.log(operate("*")(6, 4)); // 24, called immediately
```

► CONSOLE OUTPUT

10 24

7. Closures

7.1 What is a Closure?

A **closure** is formed when a function "remembers" the variables from its lexical scope, even after the outer function that created that scope has finished executing. Formally: **a closure is the combination of a function bundled together with references to its surrounding (lexical) state.**

Why do closures exist / how are they even possible? They are a direct and unavoidable consequence of **lexical scope** (Section 3) combined with functions being **first-class values** (Section 6). Since an inner function's scope chain is fixed at the moment it is *defined*, and since that inner function can be returned out of / passed out of the outer function, the JS engine cannot garbage-collect the outer function's variables — the returned inner function still holds a live reference to them. Closures aren't a separate JS feature you "turn on"; they are what naturally happens whenever an inner function outlives its outer function.

7.2 First Example

```
function makeCounter() {
  let count = 0;           // this variable lives in makeCounter's scope
  return function () {     // this inner function "closes over" count
    count++;
    return count;
  };
}
```

```
const counter1 = makeCounter();
console.log(counter1()); // 1
console.log(counter1()); // 2
console.log(counter1()); // 3
```

```
const counter2 = makeCounter(); // a brand-new, independent closure
console.log(counter2()); // 1
```

► CONSOLE OUTPUT

1 2 3 1

Key insight: `makeCounter()` has already *returned* by the time we call `counter1()`. Normally, local variables disappear once a function returns. But `count` stays alive because the returned function still references it. Also notice `counter1` and `counter2` each get their **own independent** count — every call to `makeCounter()` creates a fresh scope, hence a fresh closure.

7.3 Real-World Use Case 1 — Data Privacy / Encapsulation

```
function createBankAccount(initialBalance) {
  let balance = initialBalance; // truly private – no way to access it directly from outside

  return {
    deposit(amount) { balance += amount; return balance; },
    withdraw(amount) {
      if (amount > balance) { console.log("Insufficient funds"); return balance; }
      balance -= amount; return balance;
    },
    checkBalance() { return balance; }
  };
}
```

```
const account = createBankAccount(1000);
console.log(account.deposit(500)); // 1500
console.log(account.withdraw(2000)); // "Insufficient funds", 1500
console.log(account.balance);      // undefined - cannot reach in directly!
```

► CONSOLE OUTPUT

1500 Insufficient funds 1500 undefined

Real-world scenario: This is exactly how a real bank works — customers interact only through an ATM's buttons (the returned methods); nobody gets to walk into the vault (the private `balance` variable) directly. Closures give JavaScript a way to build "private" state long before class private fields (`#field`) existed, and this pattern is still widely used in the MERN stack, e.g. for module-level config caches or rate limiters in Express middleware.

7.4 Real-World Use Case 2 — The Classic `setTimeout` Loop Bug

```
// The classic interview question
for (var i = 1; i <= 3; i++) {
  setTimeout(function () {
    console.log("var i:", i);
  }, i * 100);
}
// Expect 1, 2, 3? You get 4, 4, 4!
```

```
► CONSOLE OUTPUT
var i: 4 var i: 4 var i: 4
```

Why? `var` is function-scoped, so there is only **ONE** shared `i` across the entire loop. All three callbacks close over that same single `i`. By the time the callbacks actually run (after the loop has fully finished), `i` is 4 for all of them.

```
// The fix: use let, which creates a NEW binding of i for each loop iteration
for (let i = 1; i <= 3; i++) {
  setTimeout(function () {
    console.log("let i:", i);
  }, i * 100);
}
```

```
► CONSOLE OUTPUT
let i: 1 let i: 2 let i: 3
```

This single example ties together **scope**, **closures**, and the practical difference between `var` and `let` — it's one of the best interview/exam questions to drill in class.

7.5 Real-World Use Case 3 — Memoization / Caching

```
function memoizedSquare() {
  const cache = {}; // private cache, kept alive by the closure
  return function (n) {
    if (n in cache) {
      console.log("From cache");
      return cache[n];
    }
    console.log("Calculating...");
    const result = n * n;
    cache[n] = result;
    return result;
  };
}

const square = memoizedSquare();
console.log(square(5)); // Calculating... 25
console.log(square(5)); // From cache 25
```

```
► CONSOLE OUTPUT
Calculating... 25 From cache 25
```

7.6 Mini Practical — Build It Yourself

```
function createStepper(step) {
  let current = 0;
  return {
    next: () => (current += step),
    reset: () => (current = 0),
    value: () => current,
  };
}

const byTwos = createStepper(2);
console.log(byTwos.next()); // 2
console.log(byTwos.next()); // 4
byTwos.reset();
console.log(byTwos.value()); // 0
```

```
► CONSOLE OUTPUT
2 4 0
```

Try it yourself: Modify `createStepper` to add a `previous()` method that subtracts the step, without adding any new top-level variables — this reinforces that the closure already gives you private, shared state.

8. Currying

8.1 What is Currying?

Currying is a technique of transforming a function that takes multiple arguments into a sequence of functions, each taking a **single** argument, where each function returns another function until all arguments have been supplied — at which point the final result is returned.

```
// Normal function - all arguments at once
function addNormal(a, b, c) {
  return a + b + c;
}
console.log(addNormal(1, 2, 3)); // 6

// Curried version - one argument at a time
function addCurried(a) {
  return function (b) {
    return function (c) {
      return a + b + c;
    };
  };
}
console.log(addCurried(1)(2)(3)); // 6
```

► CONSOLE OUTPUT

6 6

Why does currying work at all? It is only possible because of two things we already covered: (1) functions can *return* other functions (Section 6.7), and (2) each returned inner function forms a **closure** (Section 7) over the arguments captured so far. `addCurried(1)` returns a function that has `a = 1` "locked in" via closure; calling that with (2) locks in `b = 2`, and so on, until the innermost function has access to `a`, `b`, and `c` — all via the scope chain — and can finally compute the sum.

8.2 Why Use Currying? (The "Why" and "When")

- **Reusability via partial application:** lock in some arguments now, supply the rest later.
- **Configuration-style APIs:** build specialised functions from a general one.
- **Composability:** curried functions plug cleanly into functional pipelines (e.g. Redux middleware signatures like `store => next => action => {...}` are curried functions).

8.3 Real-World Scenario 1 — Reusable Discount Calculator

```
function applyDiscount(discountPercent) {
  return function (price) {
    return price - (price * discountPercent) / 100;
  };
}

const tenPercentOff = applyDiscount(10); // a specialised, reusable function
const festiveSale = applyDiscount(30);

console.log(tenPercentOff(1000)); // 900
console.log(tenPercentOff(500)); // 450
console.log(festiveSale(1000)); // 700
```

► CONSOLE OUTPUT

900 450 700

Real-world use: An e-commerce MERN app can generate one `tenPercentOff` function and reuse it across hundreds of product prices, instead of re-passing the discount percentage every single time. This is exactly the "lock in configuration, reuse the specialised function" pattern currying is built for.

8.4 Real-World Scenario 2 — Logger Factory

```
const log = (level) => (module) => (message) =>
  console.log(`[${level}] (${module}): ${message}`);

const errorLog = log("ERROR");
const authErrorLog = errorLog("AuthService");

authErrorLog("Invalid token");
authErrorLog("Session expired");
```

► CONSOLE OUTPUT

[ERROR] (AuthService): Invalid token [ERROR] (AuthService): Session expired

8.5 A Generic curry() Helper (Advanced)

Instead of hand-writing nested functions every time, we can write one generic helper that curries ANY function, regardless of how many parameters it has.

```
function curry(fn) {
  return function curried(...args) {
    if (args.length >= fn.length) { // fn.length = number of declared parameters
      return fn.apply(this, args); // enough args collected - run the original function
    }
    return function (...nextArgs) { // not enough yet - keep collecting
      return curried.apply(this, args.concat(nextArgs));
    };
  };
}

function volume(l, b, h) { return l * b * h; }
const curriedVolume = curry(volume);

console.log(curriedVolume(2)(3)(4)); // 24 - one at a time
console.log(curriedVolume(2, 3)(4)); // 24 - mixed grouping
console.log(curriedVolume(2, 3, 4)); // 24 - all at once, still works
```

```
► CONSOLE OUTPUT
24 24 24
```

Why this matters for advanced learners: This generic `curry()` is a strong "advanced" checkpoint — it combines closures, rest parameters, recursion (curried calling itself), and `Function.prototype.length`. It's a common asked in MERN/frontend interviews.

8.6 Currying vs Partial Application (Clarify the Difference)

Currying	Partial Application
Always transforms into a chain of single-argument functions	Fixes some arguments now (any number), returns a function expecting the rest
<code>add(1)(2)(3)</code>	<code>add(1, 2)(3)</code> or <code>add(1)(2, 3)</code>
Goal: uniform one-arg-at-a-time interface	Goal: pre-configure a function with known values

8.7 Mini Practical

```
// Curried validator - build reusable, specific validators from one generic rule
const validateLength = (min) => (max) => (value) =>
  value.length >= min && value.length <= max;

const isValidUsername = validateLength(3)(15);
console.log(isValidUsername("ab")); // false - too short
console.log(isValidUsername("priya_23")); // true
```

```
► CONSOLE OUTPUT
false true
```

Try it yourself: Write `curry3(fn)` that works only for exactly 3-argument functions (a simpler, non-generic version), before looking at the fully generic `curry()` from Section 8.5.

9. Output-Based Question Bank

20 questions, ordered from basic to advanced, covering every subtopic above. How to use this bank: **(1)** look only at the code, **(2)** give yourself 60 seconds to write down your predicted output, **(3)** reveal the answer and go through the reasoning.

Q1 **EASY** — var vs hoisting

```
console.log(a);  
var a = 5;
```

Output: `undefined`

Why: `var a` is hoisted and auto-initialized to `undefined` in the memory creation phase; the assignment `a = 5` only happens at execution time, later.

Q2 **EASY** — TDZ

```
console.log(b);  
let b = 5;
```

Output: `Uncaught ReferenceError: Cannot access 'b' before initialization`

Why: `b` is in the Temporal Dead Zone until its declaration line executes.

Q3 **EASY** — block scope

```
{  
  let x = 10;  
}  
console.log(x);
```

Output: `Uncaught ReferenceError: x is not defined`

Why: `let` is block-scoped; `x` never existed outside the `{}`.

Q4 **MEDIUM** — function hoisting vs var

```
foo();  
var foo = function () { console.log("A"); };  
function foo() { console.log("B"); }
```

Output: `B`

Why: In the creation phase, function declarations are hoisted *fully*, and win over a `var` with the same name (the `var foo` declaration is hoisted but effectively ignored since `foo` is already a function; the assignment to the function expression hasn't run yet at the point of the call).

Q5 **MEDIUM** — lexical scope, not call-site

```
let a = "outer";  
function first() { console.log(a); }  
function second() {  
  let a = "inner";  
  first();  
}  
second();
```

Output: `outer`

Why: `first` is lexically defined in the global scope, so its scope chain looks at the global `a` — regardless of being called from inside `second`.

Q6 **MEDIUM** — closures in a loop (var)

```
const fns = [];  
for (var i = 0; i < 3; i++) {  
  fns.push(() => console.log(i));  
}
```

```
fns.forEach(fn => fn());
```

Output: 3 3 3

Why: One shared `i` (function-scoped), so all three closures reference the same final value after the loop ends.

Q7 MEDIUM — closures in a loop (let)

```
const fns = [];  
for (let i = 0; i < 3; i++) {  
  fns.push(() => console.log(i));  
}  
fns.forEach(fn => fn());
```

Output: 0 1 2

Why: `let` creates a fresh binding of `i` for every iteration, so each closure captures its own separate `i`.

Q8 MEDIUM — shadowing

```
let x = 1;  
function test() {  
  console.log(x);  
  let x = 2;  
}  
test();
```

Output: Uncaught ReferenceError: Cannot access 'x' before initialization

Why: The local `let x` shadows the outer `x` for the whole function body, and it's still in TDZ at the point of `console.log`.

Q9 MEDIUM — closures preserve state

```
function makeMultiplier(x) {  
  return function (y) { return x * y; };  
}  
const double = makeMultiplier(2);  
const triple = makeMultiplier(3);  
console.log(double(5), triple(5));
```

Output: 10 15

Why: Each call to `makeMultiplier` creates a distinct scope and thus a distinct closure — `double` and `triple` don't share their `x`.

Q10 MEDIUM — arrow fn and this

```
const obj = {  
  val: 42,  
  regular: function () { return (() => this.val)(); },  
};  
console.log(obj.regular());
```

Output: 42

Why: The arrow function has no `this` of its own, so it lexically inherits `this` from `regular`, which is `obj` (since `regular` is called as `obj.regular()`).

Q11 MEDIUM — currying, partial calls

```
const add = a => b => c => a + b + c;  
const step1 = add(5);  
const step2 = step1(10);  
console.log(step2(15));  
console.log(step1(20)(2));
```

Output: 30 then 27

Why: `step1` is a closure with `a = 5` locked in and can be reused independently for different `b / c` combinations — each call creates a fresh inner closure without mutating `step1` itself.

Q12 **HARD** — const with a mutable object

```
const user = { name: "Ravi" };
user.name = "Kiran";
console.log(user.name);
user = { name: "New" };
```

Output: First logs `Kiran`, then `Uncaught TypeError: Assignment to constant variable.`

Why: `const` prevents **reassigning the binding** (`user = ...`), not mutating the object it points to. Changing a property is allowed; replacing the whole object is not.

Q13 **HARD** — IIFE + closure

```
const counter = (function () {
  let count = 0;
  return { increment: () => ++count };
})();
console.log(counter.increment());
console.log(counter.increment());
```

Output: `1` then `2`

Why: The IIFE runs once immediately, creating one private `count` via closure, and returns an object whose method keeps that closure alive across calls.

Q14 **HARD** — function vs block hoisting inside if

```
if (true) {
  function greet() { console.log("Hi"); }
}
console.log(typeof greet);
```

Output (in strict/module code, and modern V8): `function` — but this is environment-dependent legacy behaviour.

Note: Historically, function declarations inside blocks had inconsistent cross-browser hoisting behaviour. The safe, modern rule to teach: **always treat a function declared inside a block as scoped to that block**, and use a function expression assigned to a `let` / `const` if you need predictable, explicit scoping.

Q15 **HARD** — closures over object references

```
function createLogger() {
  const logs = [];
  return {
    add: (msg) => logs.push(msg),
    getAll: () => logs,
  };
}
const logger = createLogger();
logger.add("start");
const snapshot = logger.getAll();
logger.add("end");
console.log(snapshot);
```

Output: `[ 'start', 'end' ]`

Why: `snapshot` is a reference to the SAME `logs` array kept alive by the closure — it is not a copy, so later mutations via `logger.add("end")` are visible through `snapshot` too.

Q16 **HARD** — currying with a shared cache (closures + currying)

```
function curryAdd(a) {
  let calls = 0;
  return function (b) {
    calls++;
    console.log(`call #${calls}`);
    return a + b;
  };
}
const addTo10 = curryAdd(10);
console.log(addTo10(5));
console.log(addTo10(1));
```

Output: `call #1`, `15`, `call #2`, `11`

Why: `addTo10` is one closure instance, so `calls` and `a` persist and accumulate across multiple invocations of the SAME returned function.

Q17 **HARD** — TDZ with `typeof`

```
console.log(typeof notDeclared); // not declared anywhere
console.log(typeof inTDZ);      // declared below with let
let inTDZ = 5;
```

Output: `undefined` then `Uncaught ReferenceError: Cannot access 'inTDZ' before initialization`

Why: `typeof` on a truly undeclared variable safely returns `"undefined"`. But `typeof` on a variable that IS declared with `let` / `const` further down, while still in its TDZ, still throws — TDZ overrides `typeof`'s usual "safe" behaviour.

Q18 **HARD** — nested closures

```
function outer() {
  let count = 0;
  function middle() {
    function inner() {
      count++;
      return count;
    }
    return inner;
  }
  return middle();
}
const fn = outer();
console.log(fn());
console.log(fn());
```

Output: `1` then `2`

Why: Closures chain through multiple levels — `inner` closes over `middle`'s scope, which itself has no `count`, so the lookup continues up to `outer`'s scope, where `count` lives and persists across calls.

Q19 **HARD** — `function.length` and generic curry

```
function curry(fn) {
  return function curried(...args) {
    return args.length >= fn.length
      ? fn(...args)
      : (...more) => curried(...args, ...more);
  };
}
const vol = curry((l, b, h) => l * b * h);
console.log(vol(2)(3, 4));
console.log(vol(2, 3)(4));
```

Output: `24` then `24`

Why: `fn.length` (3, since `vol`'s original function takes 3 params) tells `curried` how many total arguments to wait for, regardless of how they're grouped across calls; closures accumulate them in `args` via recursion.

Q20 **HARD** — everything together

```
const makeAccount = (balance) => {
  const history = [];
  return (action) => (amount) => {
    if (action === "deposit") balance += amount;
    if (action === "withdraw") balance -= amount;
    history.push(`${action}:${amount}`);
    return { balance, history };
  };
};
const acc = makeAccount(100);
console.log(acc("deposit")(50));
console.log(acc("withdraw")(30));
```

Output: `{ balance: 150, history: [ 'deposit:50' ] }` then `{ balance: 120, history: [ 'deposit:50', 'withdraw:30' ] }`

Why: This combines currying (`action` then `amount`, one at a time) with a closure over `balance` and `history`, which persist and accumulate across calls to `acc(...)` because they all share the same outer scope created once by `makeAccount(100)`.

10. One-Page Recap & Cheat Sheet

Concept	One-line definition	Key gotcha to remember
Scope	Where a name is visible/accessible	var = function-scoped, let/const = block-scoped
Lexical Scope	Scope fixed by where code is WRITTEN, not called from	Scope chain looks outward only, never inward
Hoisting	Names registered in memory before execution starts	var → undefined; function decl → fully usable; let/const → TDZ
TDZ	Gap between scope start and let/const's declaration line	typeof does NOT protect you from a TDZ ReferenceError
Functions	Reusable, first-class blocks of logic	Arrow fns don't have their own `this`; IIFEs run once, immediately
Closures	A function + memory of its lexical scope	Each outer call creates an independent closure instance
Currying	Multi-arg fn → chain of single-arg functions	Powered entirely by closures capturing earlier arguments

A Mental Model to Sketch Out

Draw three concentric rectangles labelled (outside → in): **Global Scope** → **Function Scope** → **Block Scope**. Then draw an arrow from the innermost box pointing outward through each layer, labelled "the scope chain looks this way when a name isn't found locally." This single diagram ties Sections 2, 3, and 7 together visually.

11. Practice Exercises

Spend 5–10 minutes on each exercise. Write down your prediction before you run the code — that's where the real learning happens.

1. **Scope hunt:** Write a 3-level nested function where each level declares a variable with the SAME name. Log the variable at each level and explain shadowing.
2. **Hoisting trap:** Write a function that mixes a `var`, a `let`, and a function declaration, and predict console output at 3 different points before any assignment runs.
3. **Fix the TDZ bug:** Given a snippet that throws a TDZ error, reorder the declarations to fix it without changing behaviour.
4. **Build a private counter:** Using a closure, build a counter object with `increment`, `decrement`, and `reset`, with NO way to directly read/write the count from outside.
5. **Curry a discount calculator:** Write `discount(percent)(price)` and create at least two reusable specialised discount functions from it (e.g. `student10`, `festival25`).
6. **Debug the loop:** Given a `var`-based `setTimeout` loop printing the wrong values, fix it two different ways — (a) switch to `let`, (b) keep `var` but wrap the body in an IIFE that takes `i` as a parameter.

Bonus challenge: Implement the generic `curry(fn)` helper from Section 8.5 from memory, then use it to curry a 4-argument function and test all possible argument groupings.